

Transportation Module — QA Test Guide

Guest Transportation & Driver Assignment module — Postman test guide with sample data.
Base URL used below: `{{baseUrl}}` (Postman environment variable) → set to your API host, e.g. `https://localhost:7001`.

1. Setup — Postman Environment

Create a Postman environment with these variables. Test Case 0 (login) fills token automatically if you add the suggested test script; otherwise paste it manually after login.

Variable	Example value	Notes
baseUrl	https://localhost:7001	API host
token	(JWT from login)	Used as Authorization: Bearer <code>{{token}}</code> for admin calls
guestToken	(JWT from guest login)	Used for the guest-app ride-request endpoints
driverToken	(JWT from driver login)	Used for the driver-app endpoints
eventId	a real Event PublicId (GUID)	From GET <code>/api/v1/events</code>
guestId	a real Guest PublicId (GUID)	From GET <code>/api/v1/guest</code>
driverId	a real DriverProfile PublicId (GUID)	From GET <code>/api/v1/travel/drivers</code> (lookup)
rideRequestId	(filled after Test Case 5)	Ride request PublicId
transportId	(filled after Test Case 2)	Schedule PublicId

Every request below needs header Authorization: Bearer `{{token}}` (or `guestToken`/`driverToken` where noted) and Content-Type: `application/json`. Admin endpoints additionally require the calling user's role to hold the `Transportation.View` / `Transportation.Manage` / `Transportation.Assign` permissions (already granted to the seeded **Travel & Logistics Manager** role).

2. Endpoint Reference

Method	Path	Who	Purpose
POST	<code>/api/v1/transportation/schedule</code>	Admin	Create a pre-scheduled ride
DELETE	<code>/api/v1/transportation/schedule/{id}</code>	Admin	Cancel a scheduled ride
GET	<code>/api/v1/transportation/guest/{guestId}</code>	Admin	Guest's full schedule

GET	/api/v1/transportation/event/{eventId}	Admin	Event schedule board (paged)
GET	/api/v1/transportation/guest/{guestId}/drivers	Admin	Guest's assigned driver pool
POST	/api/v1/transportation/guest/{guestId}/drivers	Admin	Assign driver(s) to a guest
POST	/api/v1/ride-requests	Guest	Create an on-demand ride request
GET	/api/v1/ride-requests/mine	Guest	Guest's own ride requests
DELETE	/api/v1/ride-requests/{id}	Guest	Cancel own open ride request
POST	/api/v1/transport-app/availability	Driver	Toggle available for broadcasts
GET	/api/v1/transport-app/ride-requests/open	Driver	List open ride requests
POST	/api/v1/transport-app/ride-requests/{id}/accept	Driver	Accept a ride request (first wins)
PUT	/api/v1/transport-app/jobs/{id}/status	Driver	Advance a job: assigned→arrived→in-progress→completed

3. Test Cases — Sample Data (7)

Test Case 1 — Create a scheduled ride (success) Admin

POST {{baseUrl}}/api/v1/transportation/schedule

```
{
  "guestId": "{{guestId}}",
  "driverId": "{{driverId}}",
  "pickupLocationId": null,
  "dropoffLocationId": null,
  "scheduledTime": "2026-08-20T10:00:00",
  "notes": "Airport pickup - Driver A"
}
```

Expected: 200 OK, success: true, data.status: "assigned", data.rideSource: "scheduled". Save data.id as transportId.

Test Case 2 — Guest double-booking conflict (business rule)

Admin

POST `{{baseUrl}}/api/v1/transportation/schedule`

Repeat Test Case 1's exact body but change only `driverId` to a different valid driver, keeping the **same guest** and the **same minute** (seconds may differ, e.g. 10:00:47):

```
{
  "guestId": "{{guestId}}",
  "driverId": "{{anotherDriverId}}",
  "scheduledTime": "2026-08-20T10:00:47",
  "notes": "Should be rejected - same guest, same minute"
}
```

Expected: 409 Conflict:

```
{
  "success": false,
  "message": "This guest already has a transportation booking scheduled for 10:00 AM on 20-Aug-2026. Please select a different time.",
  "errorCode": "TRANSPORTATION_CONFLICT"
}
```

Verify: seconds are ignored — 10:00:00 and 10:00:47 must be treated as the same slot.

Test Case 3 — Driver double-booking conflict

Admin

POST `{{baseUrl}}/api/v1/transportation/schedule`

Use a **different guest** but the **same driver and same minute** as Test Case 1:

```
{
  "guestId": "{{anotherGuestId}}",
  "driverId": "{{driverId}}",
  "scheduledTime": "2026-08-20T10:00:00",
  "notes": "Should be rejected - driver already booked at this time"
}
```

Expected: 409 Conflict:

```
{
  "success": false,
  "message": "Driver is already assigned to another transportation schedule during the selected time.",
  "errorCode": "TRANSPORTATION_CONFLICT"
}
```

Test Case 4 — Assign a driver pool to a guest

Admin

POST `{{baseUrl}}/api/v1/transportation/guest/{{guestId}}/drivers`

```
{
  "driverIds": ["{{driverId}}", "{{anotherDriverId}}"]
}
```

Expected: 200 OK, data is an array of the guest's eligible drivers (name + id). Re-run GET on the same URL to confirm it persisted.

Test Case 5 — Guest creates an on-demand ride request

Guest app (guestToken)

POST {{baseUrl}}/api/v1/ride-requests

```
{
  "pickupLocationId": null,
  "dropoffLocationId": null,
  "requestedTime": null,
  "notes": "Need a ride from the hotel lobby now"
}
```

Expected: 200 OK, data.status: "Open". Save data.id as rideRequestId. If the guest already has a conflicting booking at the same time, expect the same 409 TRANSPORTATION_CONFLICT shape as Test Case 2.

Test Case 6 — Driver accepts the ride request (race test)

Driver app (driverToken)

First, toggle availability on: **POST** {{baseUrl}}/api/v1/transport-app/availability → { "isAvailable": true }.

Then confirm it's visible: **GET** {{baseUrl}}/api/v1/transport-app/ride-requests/open should list rideRequestId from Test Case 5.

POST {{baseUrl}}/api/v1/transport-app/ride-requests/{{rideRequestId}}/accept (empty body)

Expected (1st call): 200 OK, data.status: "Accepted", data.transportId populated.

Concurrency check: send the *same* accept request again (same or a second driver token) immediately after.

Expected (2nd call): 409 Conflict:

```
{
  "success": false,
  "message": "This ride request is no longer available.",
  "errorCode": "TRANSPORTATION_CONFLICT"
}
```

For a true race test in Postman: use the Collection Runner or two open tabs and fire both accept requests within the same second against the same rideRequestId — exactly one must succeed.

Test Case 7 — Cancel a scheduled ride

Admin

DELETE `{{baseUrl}}/api/v1/transportation/schedule/{{transportId}}`

Expected: 200 OK, message: "Schedule cancelled". Re-running the same DELETE returns 200 OK with "Schedule already cancelled" (idempotent) rather than an error.

4. Additional checks for QA

- Driver job status progression still works unchanged: PUT `/api/v1/transport-app/jobs/{id}/status` with body `{ "status": "arrived" }`, then "in-progress", then "completed" — each must be the single next legal step (sending anything else returns a 200 with `success: false` and an explanatory message, not a 500).
- A guest with **no** existing booking at a given time must succeed with no conflict.
- Cancelled schedules (Test Case 7) must **not** block a new booking at the same date/time for the same guest — cancelled bookings are excluded from the conflict check.
- A driver who never calls the availability toggle should **not** receive new-ride-request notifications (defaults to unavailable).